

1. Validação e Refinamento dos Métodos Propostos

A análise do script SQL e da documentação de funcionalidades sugere que a maioria dos métodos propostos está bem alinhada com o modelo de dados. As tabelas revenda, clientes, contrato, licenca, licencachave, e acessos_new (que parece ser a tabela de acessos mais recente e relevante) fornecem a estrutura necessária para suportar as funcionalidades.

Alguns pontos de refinamento e alinhamento com o script SQL são:

- A tabela revenda é a base para os métodos de revenda.
- A tabela clientes contém a razão social, nome fantasia e CNPJ, sendo a base para os métodos de cliente. A sugestão de método BuscarClientePorCNPJ(cnpj) é consistente com a coluna cnpj da tabela clientes.
- A tabela contrato possui as colunas data_inicio, data_fim e valor, que são essenciais para os métodos de contrato.
- A tabela licencachave armazena o licenca_chave e a data_criacao, que são cruciais para os métodos de gerenciamento de chaves.
- A tabela licenca (representada pelo nome de tabela licenca e a chave primária numLic) tem as colunas mac_address, chave e status, que suportam os métodos de ativação e validação.
- A tabela acessos_new é a mais adequada para o registro de acessos, pois possui uma chave estrangeira (fk_acessos_new_licenca) para a tabela licenca, o que garante a integridade dos dados e um vínculo direto, o que não ocorre em outras tabelas de acesso (acesso, acessos, acessosnew).

2. Descrição Detalhada para Cada Método

Revenda

- **Nome do Método:** AutenticarRevenda
 - **Endpoint (sugerido):** POST /revendas/autenticar
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:**
 - login (string): email ou nome_revenda da tabela revenda.
 - senha (string): senha_revenda da tabela revenda.
 - **Lógica de Execução:** Selecionar na tabela revenda onde o login e a senha correspondem. Validar as credenciais.
 - **Resposta (Output):** Em caso de sucesso, retornar um token de acesso e dados da revenda. Em caso de falha, retornar um erro de autenticação (ex: status 401 Unauthorized).
- **Nome do Método:** CadastrarRevenda
 - **Endpoint (sugerido):** POST /revendas

- **Método HTTP (sugerido):** POST
- **Parâmetros de Entrada:** dados (object) contendo nome_revenda, razao_social, cnpj, email, senha_revenda.
- **Lógica de Execução:** Verificar se a revenda já existe através do CNPJ ou email. Inserir um novo registro na tabela revenda.
- **Resposta (Output):** Retornar o objeto da revenda cadastrada com o id_revenda gerado.
- **Nome do Método:** ListarClientesPorRevenda
 - **Endpoint (sugerido):** GET /revendas/{idRevenda}/clientes
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idRevenda (int).
 - **Lógica de Execução:** Selecionar na tabela clientes todos os registros com o id_revenda correspondente.
 - **Resposta (Output):** Retornar uma lista de clientes associados à revenda.

Anagrafica (Clientes)

- **Nome do Método:** CadastrarCliente
 - **Endpoint (sugerido):** POST /clientes
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:** dados (object) contendo razao_social, cnpj, nome_fantasia, id_revenda, logradouro, cidade, estado, cep.
 - **Lógica de Execução:** Inserir um novo registro na tabela clientes.
 - **Resposta (Output):** Retornar o objeto do cliente cadastrado com seu id_cliente.
- **Nome do Método:** BuscarClientePorCNPJ
 - **Endpoint (sugerido):** GET /clientes/cnpj/{cnpj}
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** cnpj (string).
 - **Lógica de Execução:** Selecionar na tabela clientes o registro com o cnpj correspondente.
 - **Resposta (Output):** Retornar o objeto do cliente encontrado. Se não houver, retornar um erro "Cliente não encontrado" (ex: status 404 Not Found).
- **Nome do Método:** AtualizarCliente
 - **Endpoint (sugerido):** PUT /clientes/{id}
 - **Método HTTP (sugerido):** PUT

- **Parâmetros de Entrada:** id (int) e dados (object) com as informações a serem atualizadas.
- **Lógica de Execução:** Atualizar as colunas correspondentes na tabela clientes para o id_cliente especificado.
- **Resposta (Output):** Retornar o objeto do cliente atualizado.
- **Nome do Método:** ListarContratos
 - **Endpoint (sugerido):** GET /clientes/{idCliente}/contratos
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idCliente (int).
 - **Lógica de Execução:** Selecionar os contratos na tabela contrato que pertencem ao id_cliente especificado.
 - **Resposta (Output):** Retornar uma lista de objetos de contrato.
- **Nome do Método:** ListarLicencas
 - **Endpoint (sugerido):** GET /clientes/{idCliente}/licencas
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idCliente (int).
 - **Lógica de Execução:** Selecionar as licenças na tabela licenca que pertencem ao id_cliente especificado.
 - **Resposta (Output):** Retornar uma lista de objetos de licença.



Contrato

- **Nome do Método:** CriarContrato
 - **Endpoint (sugerido):** POST /contratos
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:** idCliente (int), plano (string), datas (object) com data_inicio e data_fim.
 - **Lógica de Execução:** Inserir um novo registro na tabela contrato com o id_cliente e as datas fornecidas.
 - **Resposta (Output):** Retornar o objeto do contrato criado com seu id_contrato.
- **Nome do Método:** VerificarStatusContrato
 - **Endpoint (sugerido):** GET /contratos/{idContrato}/status
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idContrato (int).

- **Lógica de Execução:** Selecionar na tabela contrato o registro com o id_contrato. Comparar a data_fim com a data atual para determinar o status (ativo, vencido, etc.).
 - **Resposta (Output):** Retornar o status do contrato.
- **Nome do Método:** RenovarContrato
 - **Endpoint (sugerido):** PUT /contratos/{idContrato}/renovar
 - **Método HTTP (sugerido):** PUT
 - **Parâmetros de Entrada:** idContrato (int).
 - **Lógica de Execução:** Atualizar a data_fim na tabela contrato, estendendo o contrato a partir da data de término atual.
 - **Resposta (Output):** Retornar o objeto do contrato com a nova data de término.

LicencasChave

- **Nome do Método:** GerarChaveLicenca
 - **Endpoint (sugerido):** POST /licencas/chave
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:** idSoftware (int) e idRevenda (int).
 - **Lógica de Execução:** Gerar uma string única (licenca_chave). Inserir um novo registro na tabela licencachave com a chave gerada, o id_revenda e o id_software.
 - **Resposta (Output):** Retornar a chave de licença recém-criada.
- **Nome do Método:** ListarChavesPorRevenda
 - **Endpoint (sugerido):** GET /revendas/{idRevenda}/chaves
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idRevenda (int).
 - **Lógica de Execução:** Selecionar todas as chaves da tabela licencachave onde o id_revenda corresponde.
 - **Resposta (Output):** Retornar uma lista de chaves de licença.
- **Nome do Método:** EntregarChave
 - **Endpoint (sugerido):** PUT /licencas/chave/{idLicencaChave}/entregar
 - **Método HTTP (sugerido):** PUT
 - **Parâmetros de Entrada:** idLicencaChave (int) e destinatario (string/object).
 - **Lógica de Execução:** Atualizar o registro na tabela licencachave para marcar a chave como entregue, e associá-la a um cliente.

- **Resposta (Output):** Retornar um objeto de sucesso confirmando a entrega.

Licenca

- **Nome do Método:** AtivarLicenca
 - **Endpoint (sugerido):** POST /licencas/ativar
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:** macAddress (string) e chave (string).
 - **Lógica de Execução:** Primeiro, validar a chave na tabela licencachave. Se a chave for válida e não estiver ativada, criar um novo registro na tabela licenca com o mac_address, a chave e definir o status como 'ativa'.
 - **Resposta (Output):** Retornar um objeto de licença ativada. Erros: Chave inválida, licença já ativada.
- **Nome do Método:** ValidarLicenca
 - **Endpoint (sugerido):** GET /licencas/{numLic}/validar
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idLicenca (int).
 - **Lógica de Execução:** Selecionar o registro na tabela licenca com o numLic. Verificar o status e a data de expiração associada ao contrato do cliente para determinar se a licença é válida.
 - **Resposta (Output):** Retornar o status da licença (ativa, expirada, bloqueada).
- **Nome do Método:** RegistrarDispositivo
 - **Endpoint (sugerido):** PUT /licencas/{idLicenca}/dispositivo
 - **Método HTTP (sugerido):** PUT
 - **Parâmetros de Entrada:** idLicenca (int), ip (string), nomeComputador (string).
 - **Lógica de Execução:** Atualizar o registro na tabela licenca com as informações do dispositivo.
 - **Resposta (Output):** Retornar o objeto de licença atualizado.
- **Nome do Método:** AtualizarStatusLicenca
 - **Endpoint (sugerido):** PUT /licencas/{idLicenca}/status
 - **Método HTTP (sugerido):** PUT
 - **Parâmetros de Entrada:** idLicenca (int) e status (string).
 - **Lógica de Execução:** Atualizar a coluna status na tabela licenca para o numLic correspondente.
 - **Resposta (Output):** Retornar um objeto de sucesso com o novo status.

Acessos

- **Nome do Método:** RegistrarAcesso
 - **Endpoint (sugerido):** POST /acessos
 - **Método HTTP (sugerido):** POST
 - **Parâmetros de Entrada:** idLicenca (int), macAddress (string), ip (string).
 - **Lógica de Execução:** Inserir um novo registro na tabela acessos_new, preenchendo as colunas data_hora, id_licenca, mac_address, e external_IP com os dados fornecidos.
 - **Resposta (Output):** Retornar um objeto de sucesso.
- **Nome do Método:** ListarAcessosPorLicenca
 - **Endpoint (sugerido):** GET /licencas/{idLicenca}/acessos
 - **Método HTTP (sugerido):** GET
 - **Parâmetros de Entrada:** idLicenca (int).
 - **Lógica de Execução:** Selecionar todos os registros da tabela acessos_new que têm o id_licenca correspondente.
 - **Resposta (Output):** Retornar uma lista de objetos de acesso.

3. Diagrama de Fluxo de Funcionamento (Textual)

1. **Início:** Um revendedor é cadastrado no sistema através do método CadastrarRevenda. Ele recebe um id_revenda.
2. **Gestão de Clientes:** O revendedor autentica-se (AutenticarRevenda) e cadastra um novo cliente através do método CadastrarCliente, associando-o ao seu próprio id_revenda. O cliente recebe um id_cliente.
3. **Contrato e Chave de Licença:** Para o novo cliente, o revendedor cria um contrato (CriarContrato) e gera uma chave de licença para um software específico (GerarChaveLicenca), associando a chave à revenda e ao software.
4. **Entrega e Ativação:** O revendedor "entrega" a chave (EntregarChave) ao cliente. O cliente, ao instalar o software, utiliza a chave e o MAC Address da sua máquina para chamar o método AtivarLicenca. O sistema valida a chave e cria um novo registro na tabela licenca com o mac_address e o id da chave, definindo o status como ativa.
5. **Uso e Validação Contínua:**
 - A cada uso do software, o sistema chama o método ValidarLicenca para verificar se a licença está ativa e não expirou.
 - O sistema também registra cada acesso através do método RegistrarAcesso, inserindo um log na tabela acessos_new com o id_licenca, mac_address e external_IP.

6. **Gestão e Monitoramento:** O revendedor pode usar os métodos `ListarClientesPorRevenda`, `ListarLicencas`, `ListarChavesPorRevenda`, e `ListarAcessosPorLicenca` para monitorar e gerenciar o uso das licenças e dos clientes.